Q12)What is Software Engineering?
Answer:
Software Engineering is the systematic and disciplined approach to the development, operation, testing, and maintenance of software.
Q13)What are the main phases of the Software Development Life Cycle (SDLC)?
Answer:
The main phases of SDLC are:
1. Requirement Analysis
2. System Design
3. Implementation / Coding
4. Testing
5. Deployment
6. Maintenance
Each phase has a specific purpose. Requirement analysis identifies what the customer needs, while design defines how the system will be built. Implementation involves writing the actual code. Testing is performed to identify and fix defects before deployment. After deployment, the software is maintained by fixing bugs, improving performance, and adding new features when required.
Q14)Explain the Waterfall Model in Software Engineering.
Answer:
The Waterfall Model is a traditional software development model in which development proceeds through a sequence of well-defined phases. Each phase is generally completed before the next phase begins.
The major phases of the Waterfall Model are requirement analysis, system design, implementation, testing, deployment, and maintenance.
In the requirement analysis phase, all software requirements are collected and documented. The design phase converts these requirements into a system architecture and detailed design. During implementation, developers write the source code based on the design. The completed software is then tested to identify errors and verify that the requirements have been satisfied. After successful testing, the software is deployed for users. Finally, the maintenance phase handles changes, corrections, and improvements to the system.
The Waterfall Model is simple and easy to understand because it follows a structured sequence. It works well when requirements are stable and clearly understood. However, it is less suitable when requirements are expected to change frequently because going back to an earlier phase can be difficult and expensive.
Q15)What is software testing? Why is it important?
Answer:
Software testing is the process of evaluating a software system to find defects and verify that it behaves according to the specified requirements.
Testing is important because it helps identify errors before the software reaches the end users. It improves software quality, reliability, security, and performance. Testing can also ensure that the software meets customer requirements and behaves correctly under different conditions.

There are different levels and types of testing, such as unit testing, integration testing, system testing, and acceptance testing. Unit testing checks individual components, while integration testing checks whether different modules work correctly together. System testing evaluates the complete application, and acceptance testing determines whether the software is acceptable to the customer.
Therefore, software testing is an essential part of software development because it reduces the risk of failures and helps deliver a reliable and high-quality product.
Q16)Differentiate between functional and non-functional requirements with examples.
Answer:
Functional requirements describe what a software system should do, whereas non-functional requirements describe how the system should perform.
Functional requirements specify the features and operations that the system must provide. For example, in an online banking application, a user should be able to log in, check their account balance, transfer money, and download a bank statement. These are functional requirements because they define specific actions the system must perform.
Non-functional requirements define the quality attributes and constraints of the system. They may include performance, security, reliability, usability, scalability, and availability. For example, an online banking application may need to respond to a user request within two seconds, support 10,000 concurrent users, and encrypt sensitive customer information.
The main difference is that functional requirements focus on the functionality and behavior of the system, while non-functional requirements focus on the quality, performance, and constraints of the system. Both are important because a system must not only provide the required features but should also provide them efficiently, securely, and reliably.